

GI from Local Radiance Transfer——适合移动端上的0.5ms 全动态全实时GI



橙子

Romances broccoli

收录于 · (黑脸) (黑脸) (尖叫) (黑脸) >

之前我在文章 [橙子：给大家看看手机上0.5ms的全动态实时GI](#) 里提了一下这个全实时GI。当时因为考虑到项目要上GDC去说，所以没有给大家公开具体的算法思路。可能让有的人觉得有些不快吧。乘着我现在有了点时间，终于可以给大家详细说说这个高性能GI是怎么实现的啦。

这个0.5ms是[高通835+](#)的数据。

就是之前这个文章



给大家看看手机上0.5ms的全动态实时GI

244 赞同 · 57 评论 文章

内容超长谨慎阅读

1. 写在前面

我会告诉大家它的算法指导思想是什么。它的演化过程又是什么，还有还有它是按着怎样的一个思路来做，从而能得到这样快的一个性能。最后还有关于我个人的一些对于[图形算法+](#)，或者说自然界的物理规律。这些对于开发一些具有创新性的东西。窝觉得还是比较有意义的。

最后最后，我会从零开始，刨析，罗列出我的一些公式的推导。以及怎么从一个[形而上学+](#)的方式来理解它！！从而，希望每个人都可以很容易的实现这样一个算法！要是社区里有大佬可以在项目里落地它，或者能帮助我进一步完善它，我会万分荣幸的！

放点对比美图：

(等我补充)

2. 移动平台+的缺陷

当我们考虑一个算法跑的快不快时，我们最直接考虑到的一件事情是这个算法的时间复杂度！对于任何一个计算机科班出身的大家来说都是很浅显的。换句话说，我们无非是在考察这个算法指令数有多少，要跑多少Cycle这件事。但是事实上是，当今的移动SOC它的[算力+](#)都是非常强悍的。我就单纯说算力这件事：

硬件平台	fp32算力
Adreno 640 （对应 高通845+ ）	900 GFLOPS+

A12（对应IphoneXR）	586 GFLOPS
Adreno 740（对应高通8Gen2）	3.68 TFLOPS+
PS4	1.84 TFLOPS
Switch	0.4 TFLOPS

其中，其实高通845 和 A12都是比较老的移动端芯片了，但是他们的算力都在Switch之上。但是实际的结果却是，Switch现在移植了大量的主机游戏，比如Crysis2, Crysis3, Dying Light 等等。都是跑的延迟管线的游戏，它们的画面表现远远秒杀了一众手游。目前手游除了几个特别头部的，大部分都是页游画面。更不用说现在最新的移动端SOC，比如高通的8Gen2苹果的A16，这样的性能怪兽了。性能已经远远超越了PS4这些上一代主机。但是为什么移动平台没有出现各类3A这些高品质的游戏。

带宽

就是，其实抛开算力，如果我们看带宽会发现。移动平台的**芯片+**带宽依然很憋屈。比如移动平台为啥做TBDR，不就是为了省电，但是省电就牺牲了性能。这也就是为什么大家做移动端开发很怕**纹理采样+**，觉得这玩意开销特别高。因为大部分时间，我们都卡在带宽上面。而高的带宽，就会带来严重的发热，所以移动平台上，我们往往跑不到特别高的频率，实际上性能就没法全部发挥出来。

特别杂乱的访存的算法，对移动端一定是不友好的！！！！

(划重点，非常重要)

3. 当今GI算法的三大流派

其实不管是现在火的Lumen，BiPT，或者是SDFGI还是Surfel GI，还是前几年的SVOGI，VXGI，还是**LPV+**。又或者是更古典的PRT。其实都可以分为三大流派

3.1 预计算类

预计算类大家做手游开发用的比较多。比如各个工作室现在爱搞的各种PRT以及它的变种，**全境封锁+**的那个GI以及它的各个变种。或者是更古老的光图，还有前两年的Local Radiance Probe，都是属于预计算类的。这类算法的特点就是**开销低，部分动态，或者完全静态**。

对于实际项目开发来说，这类需要烘焙的算法，需要大量占用美术的开发时间，往往烘焙一下就是半天，而且预计算数据是很占**包体+**的！！！！

还有它不能动，一般是场景不能动。更古老的是光源不能动

3.2 追踪类

追踪这类这几年特别火爆，特别是主机平台，都是大力跟上。这里的追踪我指的不只是硬件的**光线追踪⁺**。但凡是什么包括**Trace⁺**这种操作的算法，都应该归类到追踪类上，比如Lumen，Surfel GI，SDFDDGI，VXGI，SVOGI，这些。我都认为是基于追踪的。因为它们的**访存⁺**是**随机的**。随机的访存这件事，意味着它对移动平台是极度不友好的。

3.3 传输类

这类算法，其实已经被遗忘在历史的角落里了。比较著名的有LPV，再后来，好像没有什么算法了吧。LPV最大的问题在于它的RSM，导致它只能计算单次反弹而且无非处理遮蔽。在UE4里，默认的GI算法就是LPV。之前传说是因为SVOGI开销太大了，所以最后只剩了LPV，其次它的画面表现太寡淡了。不是很值得。

4. GI算法对画面的贡献

其实大家都很喜欢讨论算法本身，讨论算法的数学原理，但是我其实更想从美术想要什么的这个视角出发来解构一下，GI对画面的贡献到底是什么。从这个角度出发，在实际游戏工业中，我们可以忽略一些数学上的正确性和精度，去往所谓的视觉友好的角度去凑。我们可以知道什么才是GI对画面真正有贡献的地方，我们就需要把更多的精力画在这上面。总结起来其实就两个点：**把画面压实，让画面柔和。**

这其实对应于GI计算我们解构出的两个成分：

各类的遮蔽，尤其是天光遮蔽

反弹的Color Bleeding

我们看到很多移动端的GI算法都把精力放在了反弹上而忽略了遮蔽。这样的GI其实是没啥用的。我给大家看几个对比图：

GI全关



没有开启GI的场景

GI只有反弹



GI只有反弹

GI只有遮蔽



GI只有遮蔽

GI On



完整的全局光照

可以看到，单用LPV或者各类VPL算法我们可以得到只有反弹的GI效果。但是画面显得很平。同样的，某些算法在移动平台上做阉割，往往算不起遮蔽，也很容易得到只有反弹的GI效果，**这样的手游容易变成页游画面**

如果手机硬件支持**光追+**我们可以得到一个简单的天光遮蔽得到近似的只有遮蔽的画面效果。但是会显得很阴沉。有点近似于早些年主机游戏的画面观感。

只有GI成分完整的GI才是真正对画面的贡献大。

5. GI from Local Radiance Transfer（进入正题）

那么怎么样做一个高性能的实时GI，同时又拥有完整的GI成分呢。

5.1 算法的思路脉络

首先，基于追踪的这类算法是肯定不可行了（这句话仅有效到2025年，再过几年，硬件跟上后，基于追踪的算法可以很容易移植上手机）。其次，我们不要各类预烘算法，什么PRT，光图这类，我们统统不考虑，因为这类算法要么**光动**不了，要么场景动不了，而且需要大量烘焙时间。我不是反对烘焙，我反对大量的巨量的烘焙。那么其实只剩一类算法可以考虑，就是基于传输的GI算法。所以这里我给它起名叫**Local Radiance Transfer**。**局部辐射传输**。为啥不传输**辐照度**呢，因为那个太糊了，移动平台本身各种东西就吃紧，没必要传输一个更低精度的数据。

5.2 LPV和光能传递的局限

那么，我们最先考虑的就是LPV这类，但是很不幸，它的表现非常差。随便看几个LPV的例子：



画面真的非常恐怖

就是它只做了简单的一次反弹，而且没有考虑到遮蔽，导致画面异常诡异。

然后我们再说一类很容易被大家无视的算法，叫光能传递。可能很多人甚至都没有听说过。但是我敢赌大家应该都玩过，Minecraft。嗯嗯。就是**minecraft**！！！！

不知道大家有没有注意过，MC里这些黑漆漆的AO是怎么计算出来的。而且MC是有手机版的，它在非常垃圾的手机上都能运行这种算法计算出大片的天空遮蔽！！这就是光能传递。



MC的光能传递制造的假遮蔽

它虽然物理不正确但是它很快

在MC的官网上有详细介绍MC的光能传递算法，本质上就是格子按能量去衰减，这个算法可以用一个嵌套的卷积操作来表示。[bugs.mojang.com/browse/...](https://bugs.mojang.com/browse/)

5.2 元胞自动机⁺

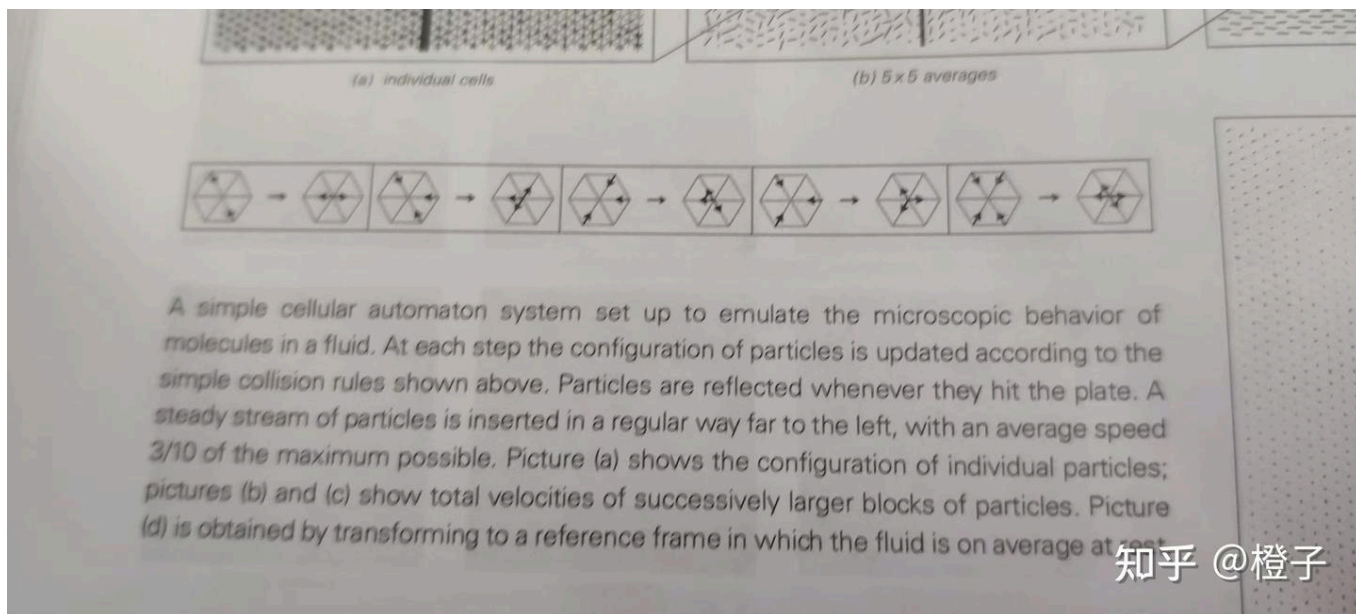
我太喜欢元胞自动机了，所以我一定要在这里给大家安利一下。它就是一个kernel一直卷啊卷。但是真的可以做很多有意思的事情。

大家来看我的这个ShaderToy，展示了一个元胞自动机

<https://www.shadertoy.com/view/XdVBWG>
www.shadertoy.com/view/XdVBWG

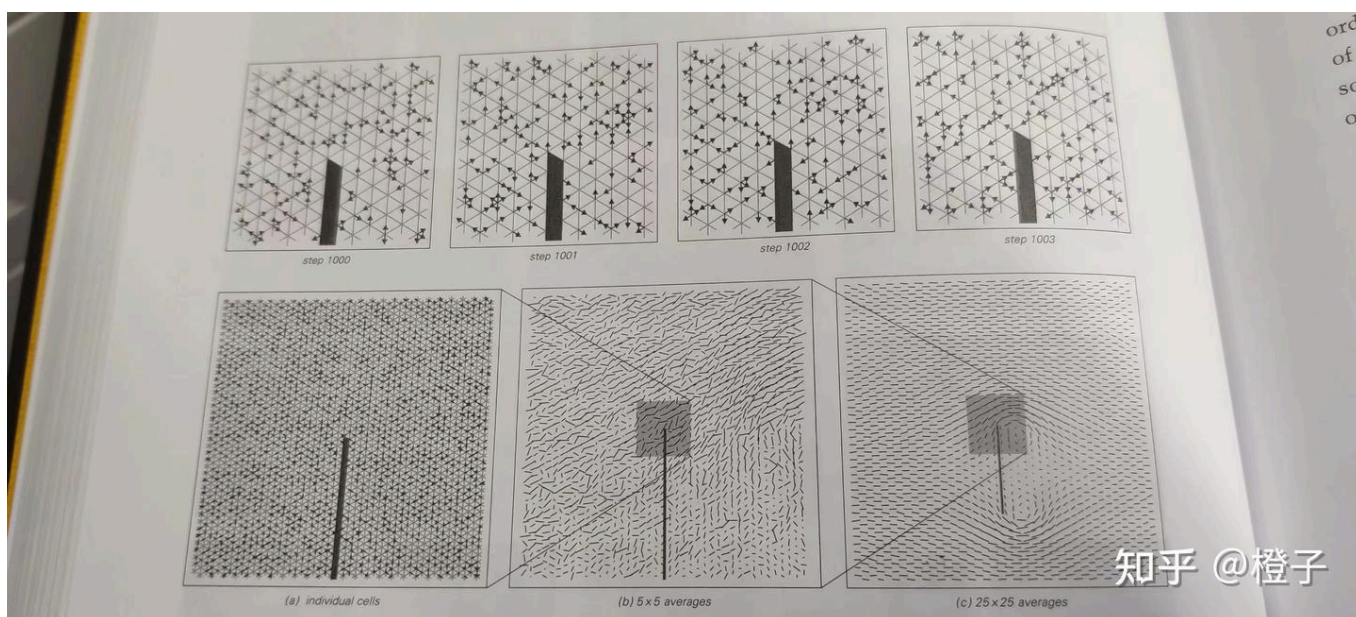
元胞自动机还可以用来做流体模拟！！ 在《A New Kind Of Science》这本书里，提到了用元胞自动机做流体模拟的方法，我给大家翻一下：

首先就是定义一些领域内的算子：



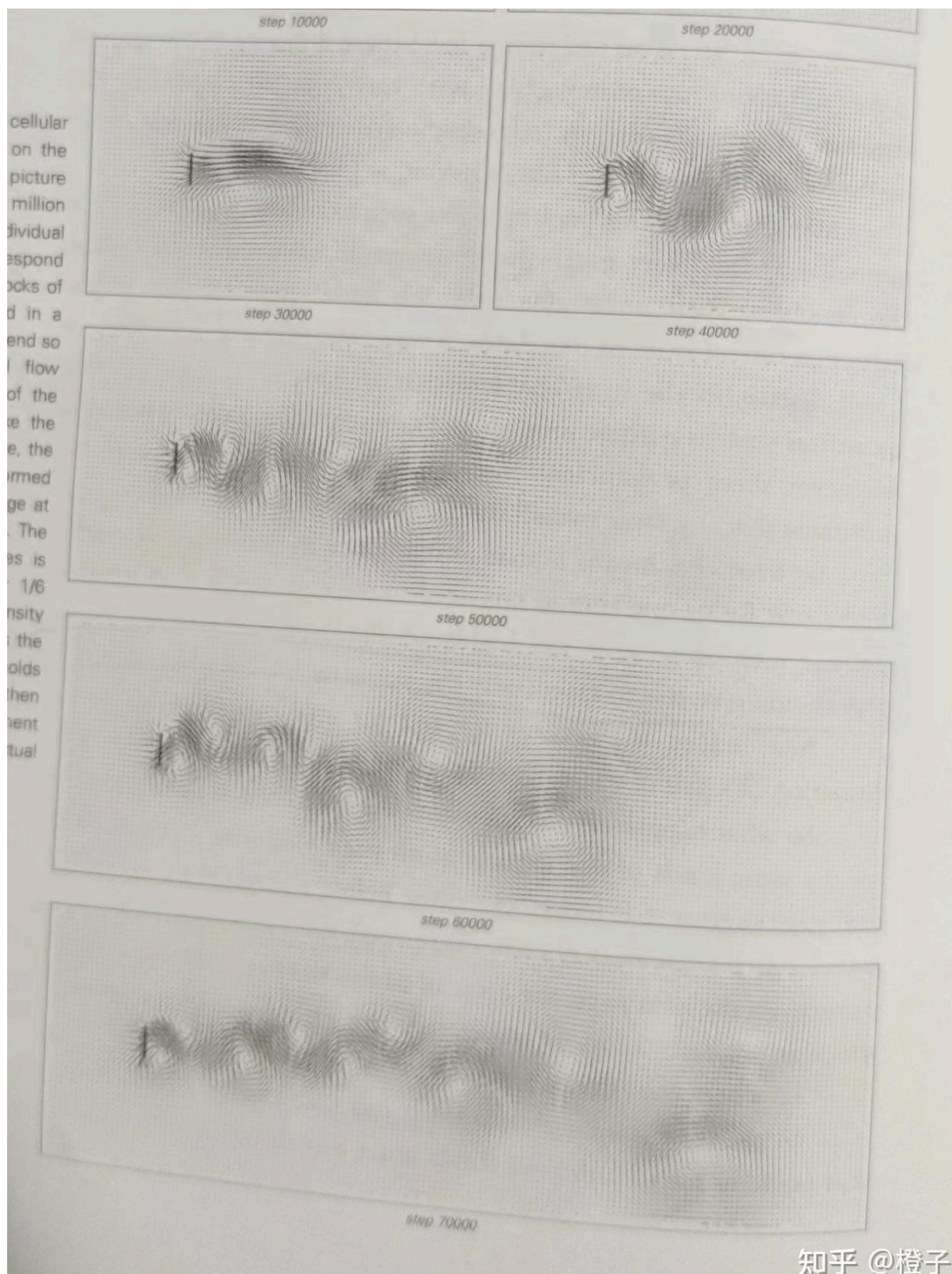
胞胞机的流体模拟算子

有了这个我们就可以把它卷积起来：



卷积了一点的胞胞机

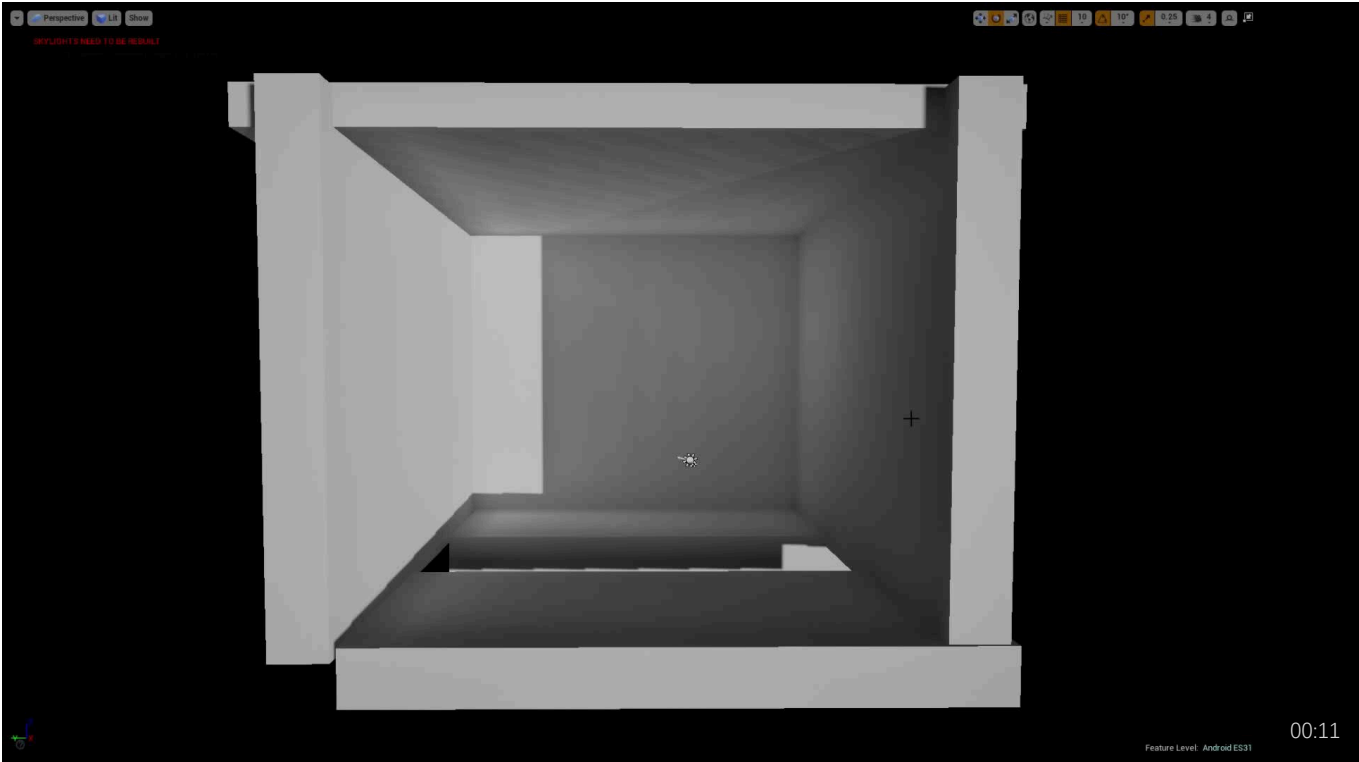
在更大尺度上的结果，可以看到已经有流体的样子了



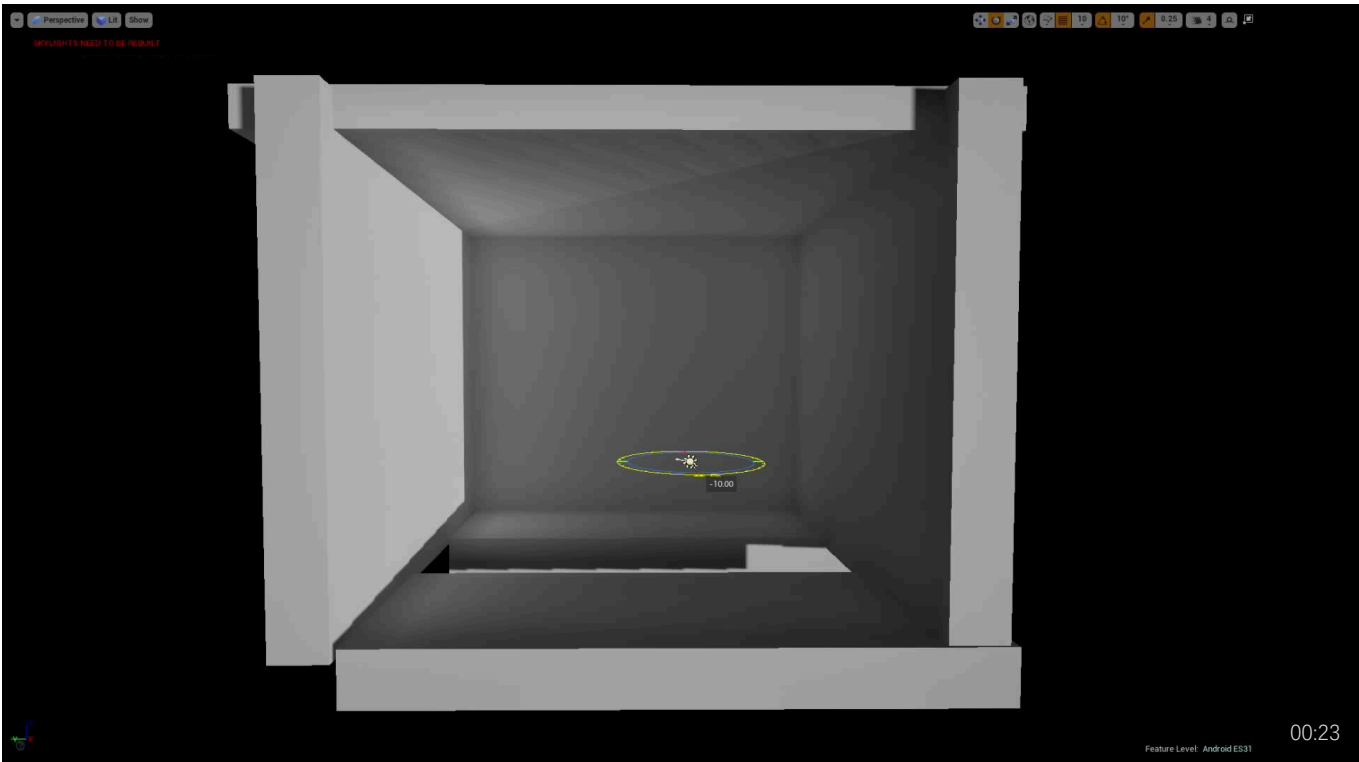
胞胞机的大尺度流体模拟

有没有发现，其实可以转化为位运算*(#O')!!!

为啥要说胞胞机呢，因为我这个算法就是从胞胞机出发的。它包含了胞胞机所具有的一切性质，也可以叫它胞胞机GI，甚至因为这种性质。我们可以看到这个GI带有的光的波动性!!，比如干涉条纹，和光波。给大家看两个早期视频，啾!!



胞胞机GI的干涉条纹欣赏



胞胞机GI中展现出的光波动性

(那个你们不要吐槽我民科⁺，虽然我真的有点民科的气质，但是我后面会给出好看的数学推导的，搓手手)

这里，如果有对胞胞机感兴趣的，我成立了一个胞胞机教，以及胞胞机教的进阶版，永恒回归⁺教。它们都是从胞胞机出发来探索和理解世界的本源的！！我是大教主⁺（尖叫）！可以加入我教！！

5.3 LPV+光能传递

这里讨论的不是简单的加法，是从这个路子出发，我们去设计一套物理正确（这个正确是指在算力无穷的情况下，随着算力的衰减这个准确度会下降）的新的GI算法⁺。

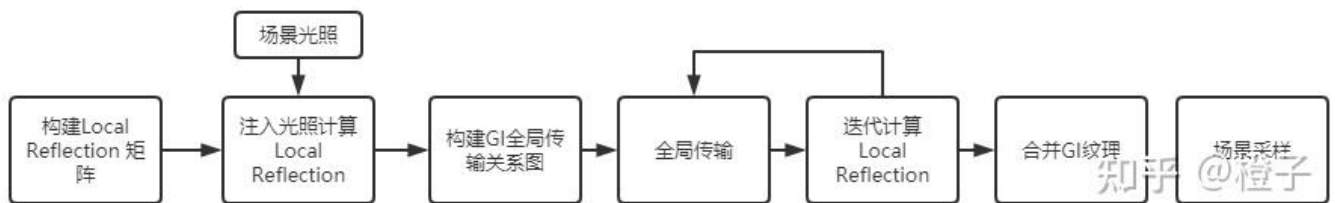
先考虑LPV的缺陷：

1. 需要RSM
2. 只能计算单次反弹
3. 没有遮蔽

再考虑光能传递的缺陷：

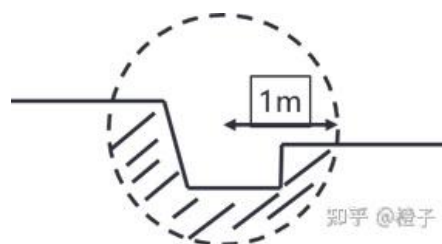
1. 物理不正确
2. 没有反弹

接下来我要开始介绍我的GI算法惹，首先我会先引入一系列新的概念。算法的大致流程如下：

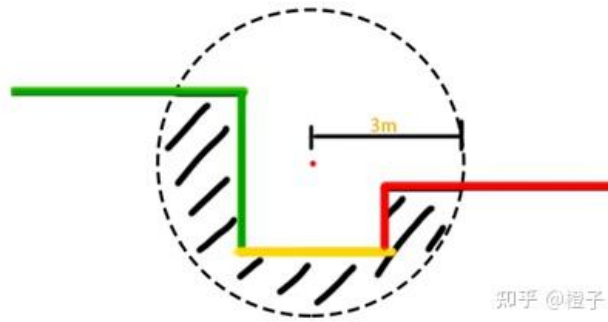


5.4 局部空间和局部传输算子

咳咳这两个是我自己起的名词，它的定义是这样的：

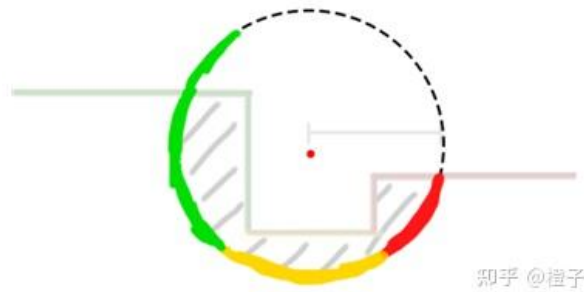


局部小空间的定义



我还画了一个有颜色的，更早的时候，那时范围更大

即在某一个小空间内，如上图1米半径的空间内。我们去考察它的GI性质。更进一步的，我们可以把这个复杂的空间内的反弹操作近似成对一个这样的球空间的操作：



近似的球空间GI

那么在这个空间下，计算GI是不是方便多了！我把这个空间里计算的GI称作Local GI。进一步的，我们可以把这个球用几个SH表示（SH不熟悉的自己去学啦，接下来全是SH），它们分别是SH_BaseColor, SH_localVisibility。考虑一个GI的迭代函数⁺：

$$L_{out} = G(L_{in}, SH_a, SH_b)$$

局部小空间的GI迭代示例

其中L_in 为进入到Local GI的光照，不管它是啥，L_in 和 L_env 等L项全部是SH，这里的全部是SH。如果SH_a为该小区域的basecolor（划掉，其实是DiffuseColor，抠字眼好累，大概就是那个意思）。SH_b为该小区域的可见性，那么会有式1：

$$L_0 = G(L_{env}, SH_{basecolor}, SH_{localvisibility})$$

式(1)

L_0 为该Local GI的第0次反弹光，L_env为照入这块Local区域的直接光。同时还会存在一个迭代球的Local的第i次反弹：

$$L_i = G(-L_{i-1}, SH_{basecolor}, SH_{localvisibility})$$

式(2)

那么对于L_localGI 有：

$$L_{LocalGI} = \sum_{i=0}^{i=n} L_i$$

式 (3)

对于GI迭代函数。

$$G(L_{in}, SH_a, SH_b) = \oint_j B \circ j \cdot \int_{\Omega} B \circ \theta \cdot Trpd(SH_a, SH_b) \cdot DFT(\theta, f_r(j, \theta)) \cdot L_{in} \cdot \cos \theta_{\rightarrow j} \, d\theta d_j$$

式子(4)

其中B为对方向j的SHBasis算子，Trpd(SH_a, SH_b)为对SH_a和SH_b的Triple Product，DFT(theta)为求解SH在方向theta上的Diffuse传输系数，f_r(j, theta)为brdf函数，由于Trpd(SH_a, SH_b)对于固定的Probe始终唯一，所以可以预计算。

我们可以看看这样一个简单的Probe，啾！！按照上面的思路做一个简单的小球，随手做一个，它对光照有怎样的相应：



好像有点大了，还有点糊

为什么放不了GIF。好讨厌啊！

可以看到，即使没有全局GI，单是局部的GI已经能够较好地反应墙角这种区域的反弹光效果。

我们上面这个过程，如果可以光追一个小空间内的小小PRT效果会更好！因为范围非常小，小于单个物件内，只需要针对逐个物件做预计算就可以了。如果是粗的实时计算也是跑的动的。

5.5 Local Transfer Matrix的全局视野

到了这里，大家可能会觉得这玩意，不就是PRT吗？嗯嗯，它确实某个角度上看是一个PRT但是加了两个约束，一个约束是小范围，第二个约束是小范围内的简化。这样有怎样的好处呢？

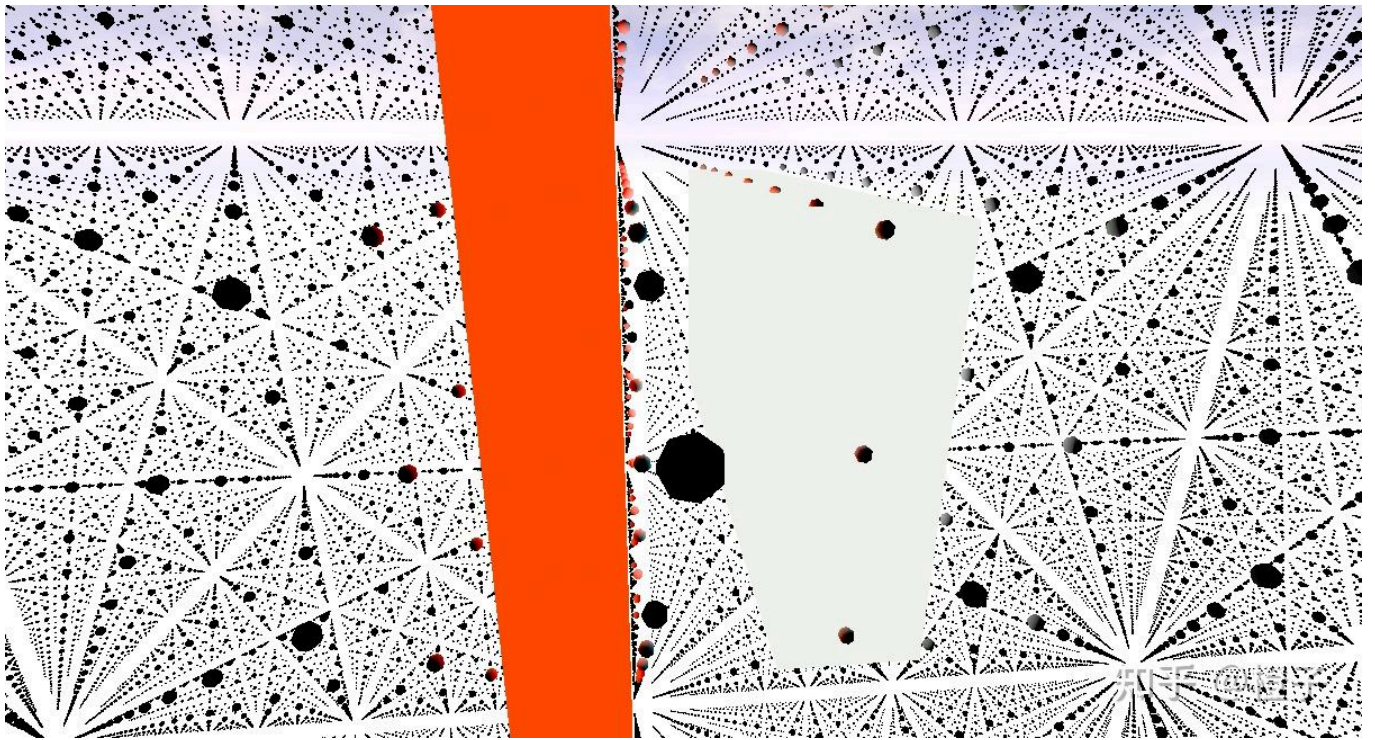
这里有一个红色的Cube和一个白色的Cube。它们的所有布满空间中的Local Transfer Matrix如下：



红色Cube 和 白色Cube 以及它们的Local Transfer Matrix

可以看到，Local Transfer Matrix只有在物件附近才是有值的，其余部分为全0！！！！

这里展示的不是反弹，只是单纯的Local Transfer Matrix。同时由于这种Local Transfer Matrix是只和领域相关的，它完美的规避了PRT计算量巨大的问题。求解一个Local Transfer Matrix只需要考察它周围的领域，甚至是一个体素化场景的周围26个点的直接访存。这样的访存是极度Cache友好的



在物件外，Local Transfer Matrix为全0

所以这些小球球都是在CPU上实时构造的！！

我们再来看看局部遮蔽的情况：

同样的，局部遮蔽只有在物件附近有值，远处为全1



局部遮蔽只有在物件附近有值

PRT的问题就在于预计算的Probe是全局相关的，导致了两个缺陷，一个是Probe的烘焙极慢。第二个是场景不可变，因为Probe是全局性的。这里考察的全部都是局部的信息，信息密度足够低所以全部的数据都能够被实时计算，而且场景随意变化。而且由于这种局部性，内存上可以做巨量的压缩。

其他的如SDF，它的Global SDF构建非常缓慢（在移动端上），也是由于它的全局性。但是Local SDF的使用依赖于Trace访存不友好。当然SDF的使用本身也需要Trace，所以访存也不友好。

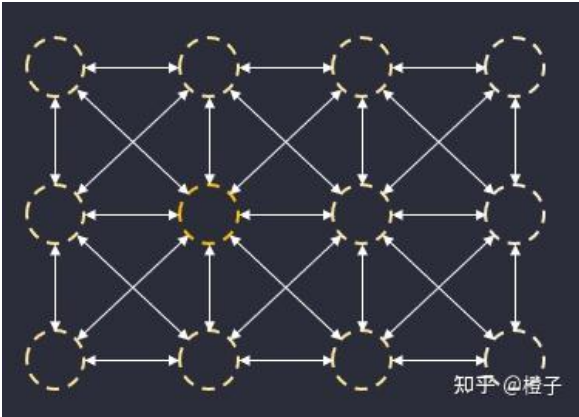
目前，我们只靠这些廉价的局部信息只能得到一个类似VPL效果的LocalGI，并不能得到一个漂亮的全局GI。接下来我讲怎么把这个局部GI变成一个物理正确的全局GI。

5.6 全局可见性传输

在做GI的时候，我们有一个开销比较高的问题就是所谓的天光遮蔽。其实就是天光的阴影。这个东西会不太好算而且需要巨量的Trace，但是在现在的这个体系下，计算天光遮蔽就会变得异常简单。我们只需要把局部的可见性上传到GPU上，使用AB Buffer通过一个简单的卷积操作来迭代出来就可以了：

对于每个Probe，每帧只考虑它周围临界26个Probe的可见性，并统计它们的可见性保存为自身的可见性。可以理解为就是在算Bloom⁺。。。

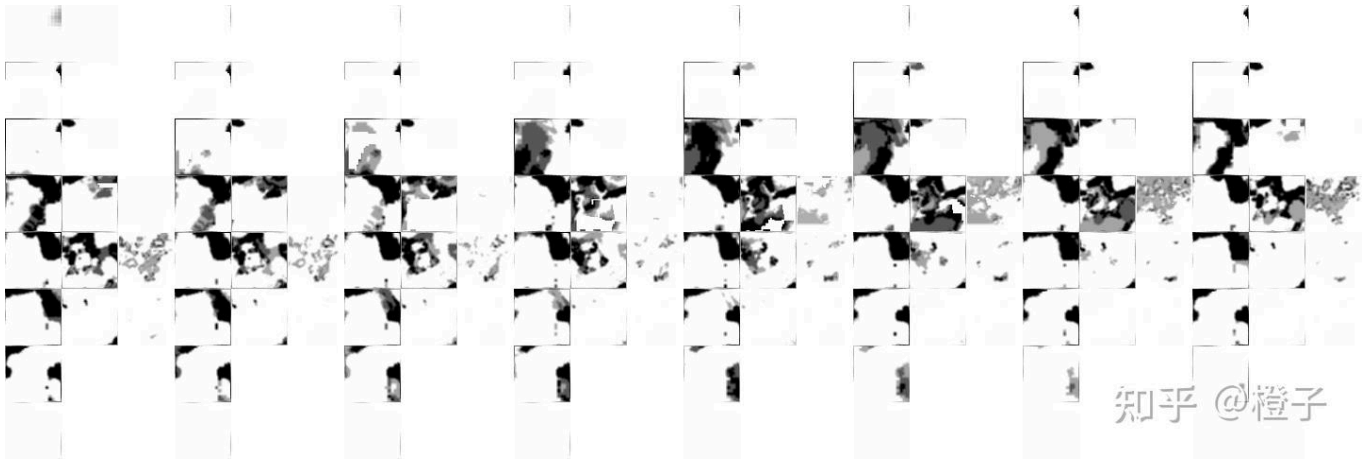
（可以无限分帧）



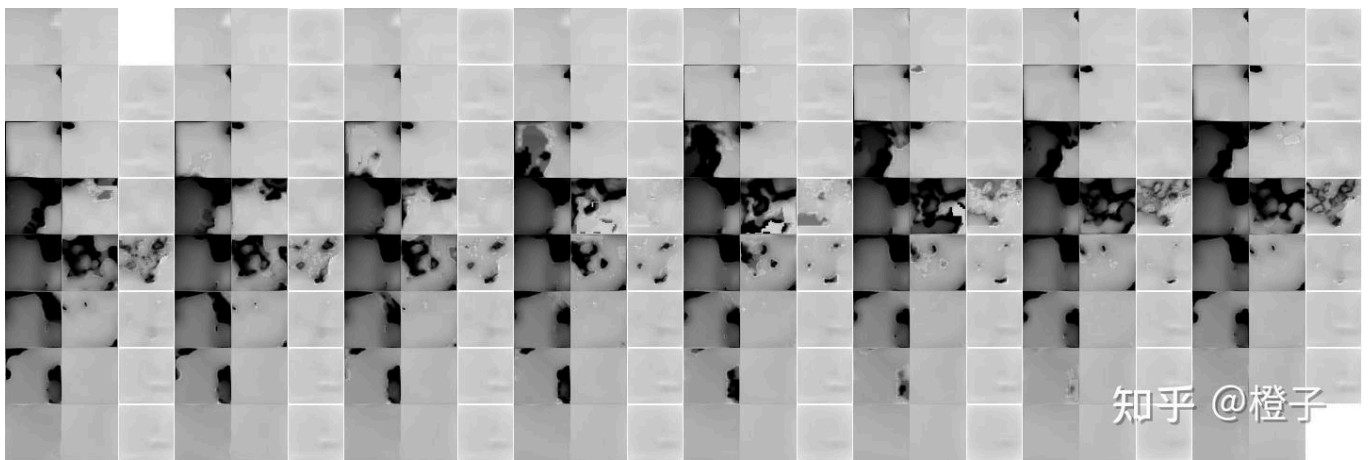
全局可见性传输

这样随着时间的迭代。我们的可见性就可以用特别远的地方传递到另一个特别远的地方，从而局部可见性变成了全局可见性。这个操作还有一个好处就是它的访存非常友好，不需要任何Trace只需要考虑它的邻接关系，我们就可以进行可见性的传输，而且这个可见性是带方向的。

原谅我有点懒，做动图真的太麻烦了（等我录个视频放上来好了），我给大家看下载传输完的全局可见性长什么样。



局部可见性，只显示SH的第0阶系数



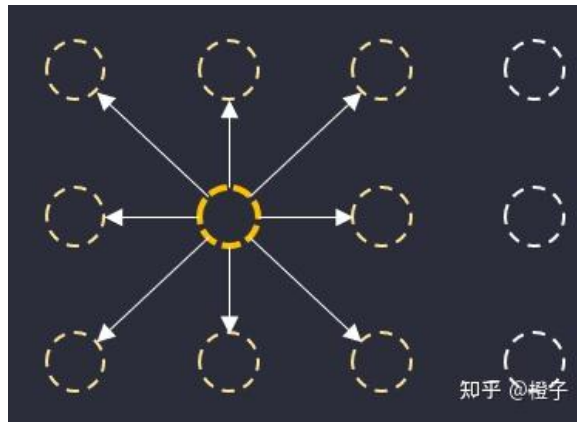
传输后的全局可见性，只显示SH的第0阶系数

因为SH的第0阶系数和AO就是一个倍数关系，这里我只展示了SH的第0阶系数。这里的数据都是体纹理平铺来展示的，可以看到纯黑的地方就是物件内部，但是物件的遮蔽被传输了出去。而且是带着方向的。

5.7 全局光照传输

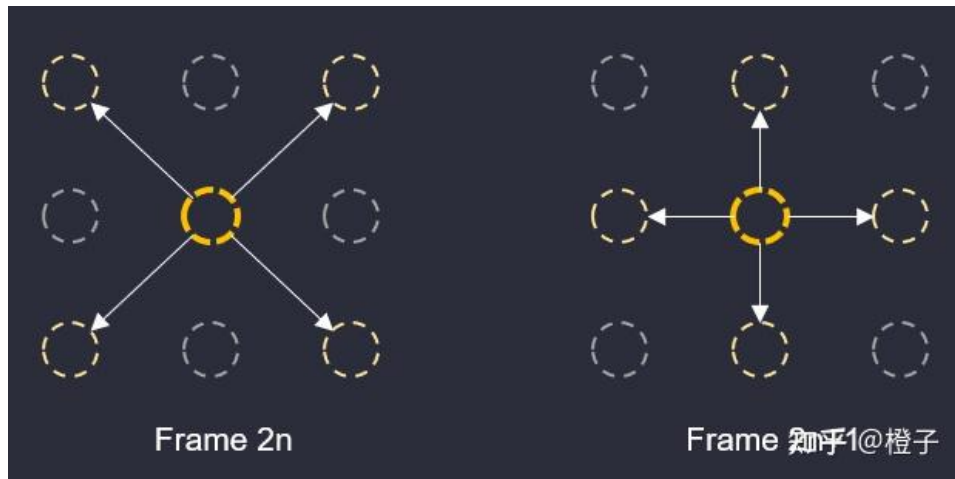
在上面，我们已经完成了全局可见性的传输，这份数据主要用于做天光遮蔽以及天光的反弹。其实天光可以更全局光照传输放在一起，这样可以进一步节省性能，但是实际的测试下来会发现，经过传输的天光，方向性会有比较大的削弱，毕竟为了性能，实际使用的是2阶的SH，它对方向性的表达能力和保持能力有限，经过较长距离的传输，天光的方向性基本丢失了，因此把天光遮蔽单独提了出来，只有天光的间接光照部分参与传输。

Radiance Transfer的过程其实和可见性的传输非常的像，主要就是做Bloom，但是其中会有一些trick来加速这个计算过程。它的计算就是每个Probe 去访问周围的26个probe获取它们的光照信息，并把它们拿过来作为自己Local 空间的入射光照：



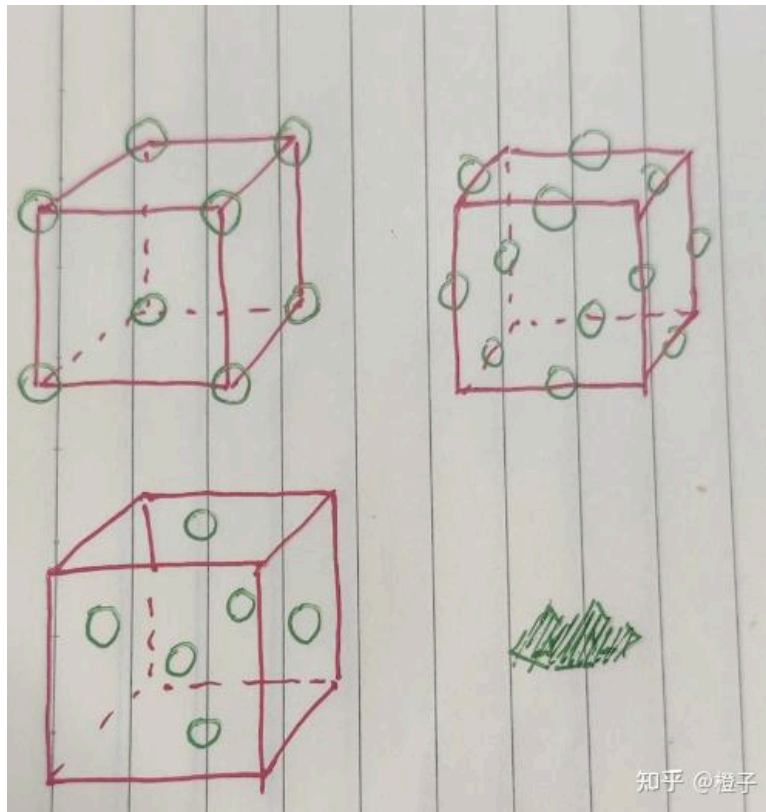
Probe GI Gather

为了进一步减少实时运算的性能开销。我们可以对这个操作进行分帧，在二维空间中的Probe传输，我们可以分为两帧来做间隔传输：



Checkboard 传输

在三维空间中，传输的patten变得比较复杂，如下图所示：



3维空间下的出传输patten

实际上跑这3组patten的实际开销还是太高了。在实际测试发现，只跑边上的这组产生的GI效果和全跑基本类似，同时加入dither可以完全类似跑3组的效果，因此实际上我只跑了边上的这组，并把它拆到3帧里，类似于3个四面体的顶点。所以我们只需要采样临接的4个probe。

接下来是光照注入。我们的光照分为面光源+和解析光源两部分。解析光源我们又把方向光单独独立出来，因为它没有具体范围的，而点光源以及Spot Light 以及面光源我们单独一类。整体的传输流程如下：

```
// 这个流程可以被独立优化一下，我这里伪码只说大体的思路
// 比如 Mesh Light 和 Local Light的注入都可以优化一下。
foreach probe
{
    SHVector InComingLight;
    if(probe not in Shadow)
    {
        InComingLight += DirectionalLightSH;
    }
    if(probe in MeshLight)
    {
        InComingLight += MeshLightSH;
    }
    if(probe in LocalLight)
    {
        InComingLight += LocalLightSH;
    }
}
```

```

// Gather 26 Direction SH Incoming Light
for(otherProbe in Direction26)
{
    InComingLight += Trpd(otherProbeSH, -otherProbeViSH);
}

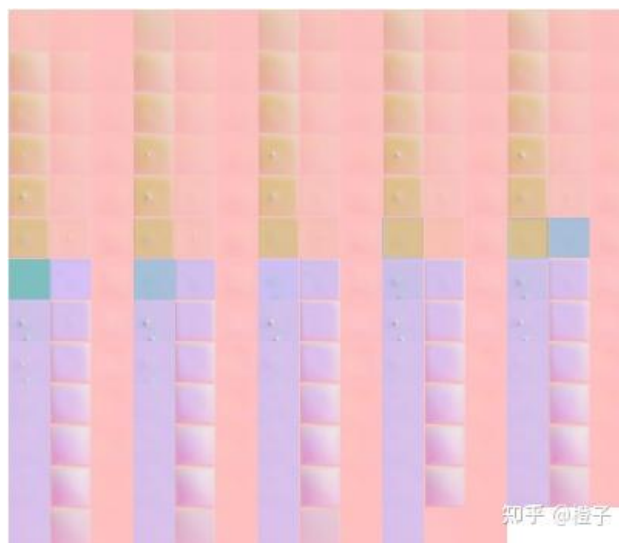
// than apply local visibility
InComingLight = Trpd(InComingLight, probeViSH);

// than do local reflection
probeSH = InComingLight * LocalTransferMatrix;
}

```

单看伪码，这个算法真的非常非常的简单。就这样我们就可以完成GI的光照传输流程，而且随着时间的推移它会变成Inf Bounds。其中的Trpd就是SH中的Triple Product，直观理解过去就是做mask。

经过上述的传输过程，我们可以得到传输后的GI体纹理：

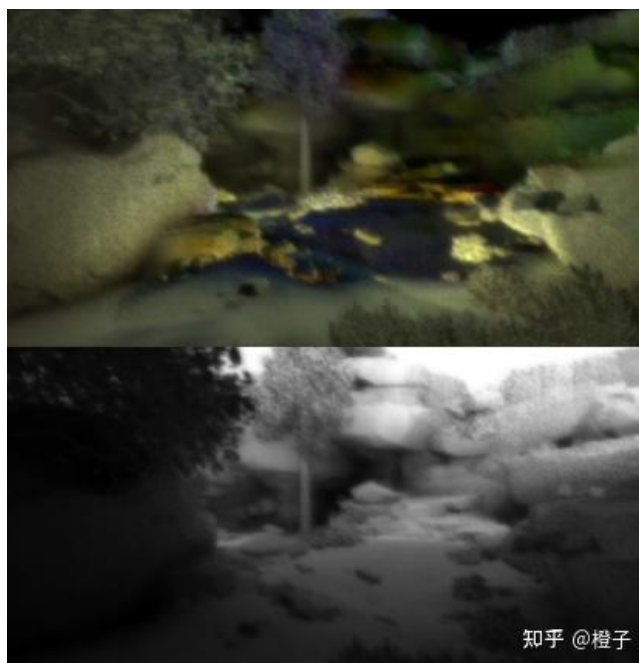


GI 传输后的体纹理

接下来我们讲怎么把这些效果合成我们最终Basepass用到的GI 数据。

5.8 Screen Space Gather

这个ScreenSpaceGather不同于Lumen的Screen Space Gather，只是用于减少Basepass对体纹理访存的开销。它就是一个低分辨率的屏幕空间pass，通常为渲染分辨率的25%。



Screen Space Gather Pass的产物

其中的RGB通道就放反弹，A通道放单独的天光遮蔽。最后我们只需要在Basepass里采样这张图就可以了！

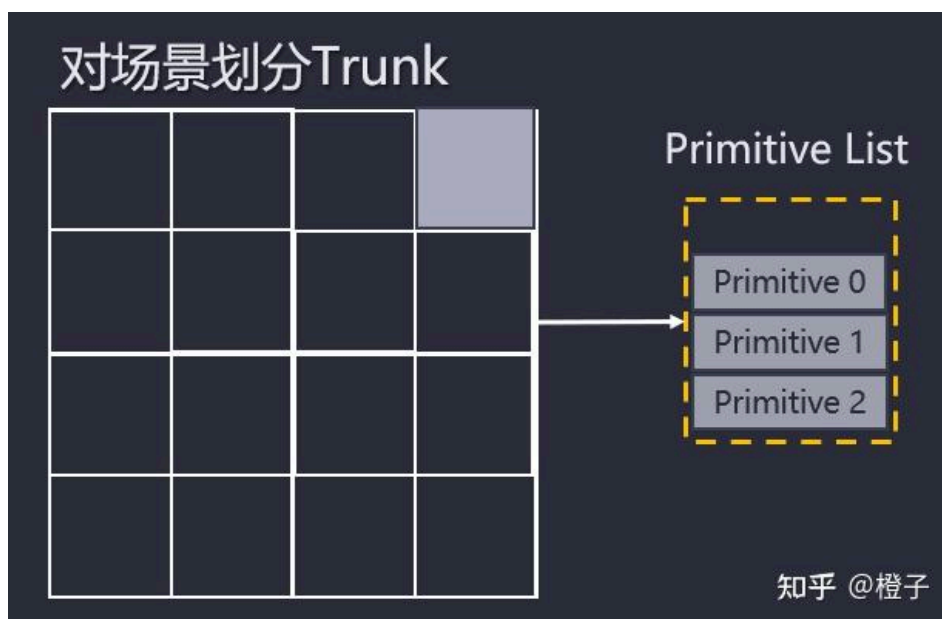
5.9 CPU侧的场景管理

由于在GPU上做传输，因此CPU上就需要去管理整个GI Scene以及，Probe的构建。在上文中，我们说过Local Transfer Probe的构建是不需要Trace的，它的构建由周围的26个邻接体素构成。接下来我就来详细说说，这个GI Scene是怎么管理的。

首先，我们需要维护一个最基本的场景管理结构。这个结构需要能快速的查询某一个空间内的GI Primitive⁺。在具体的算法实现中，我使用了一个Grid的划分来存储这些GI Primitive的指针。每个Grid的格子我称作Trunk。那么每个Trunk包含如下信息：

- Trunk覆盖范围内的 GI Primitive List
- 周围邻接Trunk的指针（用于衔接快速寻址）
- Cache

在场景管理中，我们添加和删除GI Primitive时，会先进行一个粗粒度⁺的划分，这个粗粒度的划分会快速的把一个物件划分到它对应的Trunk里，并对Trunk置脏。脏的Trunk意味着它对应的Cache失效，需要重新更新。那么实际上一个GI Primitive可能会被多个Trunk包含，比如它可能有比较大的BBBOX。具体的结构就如下图所示：

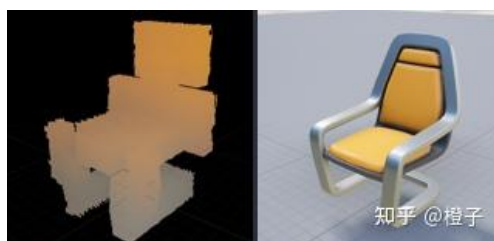


Trunk的场景划分

那么GI Primitive需要包含哪些数据呢？目前有3大类，它们分别是Color SDF，Height Field Data，Precomputed local transfer Matrix。对于一个GI Primitive来说，它同时只需要其中一种数据，就可以进行GI计算了。未来其实可以考虑如Lumen这样的Mesh Card更高效的逐物件的GI数据表达。

Color SDF:

Color SDF是由两份数据构成，一份是一个逐物件的Local SDF数据。另外一个物件的粗粒度体素化的Color数据。Color SDF的存储开销一般控制在一个物件8KB左右。



一个简单的Color SDF示例

这种数据也通常会用在SDF DDGI这样的算法中。

Height Field Data:

这个用来处理地形，它包括地形的高度图数据，以及它的极低分辨率基色数据。

Precomputed Local Transfer Matrix:

这个是我们算法独有的逐物件GI数据。它的内容和前面提到的Local Transfer Matrix一模一样。但是它的精度更高。这个更高精度不是指它的数据更多。而是因为它可以在离线构造获得更高的精度。更高的精度包括更细腻的局部遮蔽及反弹信息。它能够把物件表面的一些法线细节也考虑在内，以及物件空间内的更小范围内

的更复制遮蔽和传输关系。在离线构造这个矩阵时。我们可以用Trace来构造。并且由于Local Transfer Matrix的线性性质。我们可以在运行时直接拼接成Global 空间的Local Transfer Matrix。

以上的3种数据结构都是为了构造出我们的Local Transfer Matrix。

5.10 CPU侧的Local Transfer Matrix构造

在上文中，我反复提到所谓的Cache友好这件事。因为Cache友好我指的不只是更少的访存，它同时意味着更临近的访存。我希望的是整个GI算法中，从头到尾都没有Trace (Ray⁺ Cast) 这个操作。那么怎么用一个Cache极度友好的方法来构建Local Transfer Matrix？

首先，我们已经拥有了一个Grid划分的GI Scene。在这个Scene里，在目前Mobile平台上，我假定一个Trunk内由8x8x8个Probe组成。那么每个Probe的位置都是完全已知的，以及它要查询的GI Primitive列表也是已知的。对于一个Probe来说，目前我需要它周围的26个邻接信息来构造它的Local Transfer Matrix（实际可以更少）。那么这些邻接信息就完整的布满了整个Trunk空间。那么比如Color SDF这种数据：我们只需要把对应左边，变换到GI Primitive的Local 空间，然后即可直接访存Color SDF来获取某个位置的信息。对于Height Field Data也一样。对于Precomputed Local Transfer Matrix也一样。它们的区别只在于合成Local Transfer Matrix的操作。

对于Color SDF和Height Field Data来说，相当于做了个快速的体素化，我们需要知道26个点的邻接体素信息，然后直接用我上面给的代码即可转为Local Transfer Matrix。

对于Precomputed Local Transfer Matrix而言，则不需要访存这么多的信息，它只需要得到当前位置的信息并把它当作当前位置的Local Transfer Matrix，如果有多个覆盖，相加即可。对于Local Visibility，做Triple Product即可。

至此，我们已经得到了每个Trunk内的Local Transfer Matrix。这些数据构造了我们整个的GI Scene。接下来只需要把有值的Trunk数据上传到GPU做传输即可。

5.11 Local Transfer Matrix的自发光⁺

局部传输矩阵的特殊性，参考5.5，局部传输矩阵的构造，只需要给到Diffuse颜色。在局部传输矩阵的平坦数学空间下，它可以非常顺滑的实现从非自发光到自发光的变化。对于Diffuse反射率我们认为它小于1是吸收部分能量，等于1是完全反射能量，而大于1则表示它会发出能量，因此在构造局部传输矩阵时，我们能够非常精巧的实现GI的自发光效果，只需要把构造的Diffuse大于1即可，那么这个矩阵就是在发出能量的。我们可以看下发出能量的GI是什么表现



局部传输矩阵能量 >1 的情况

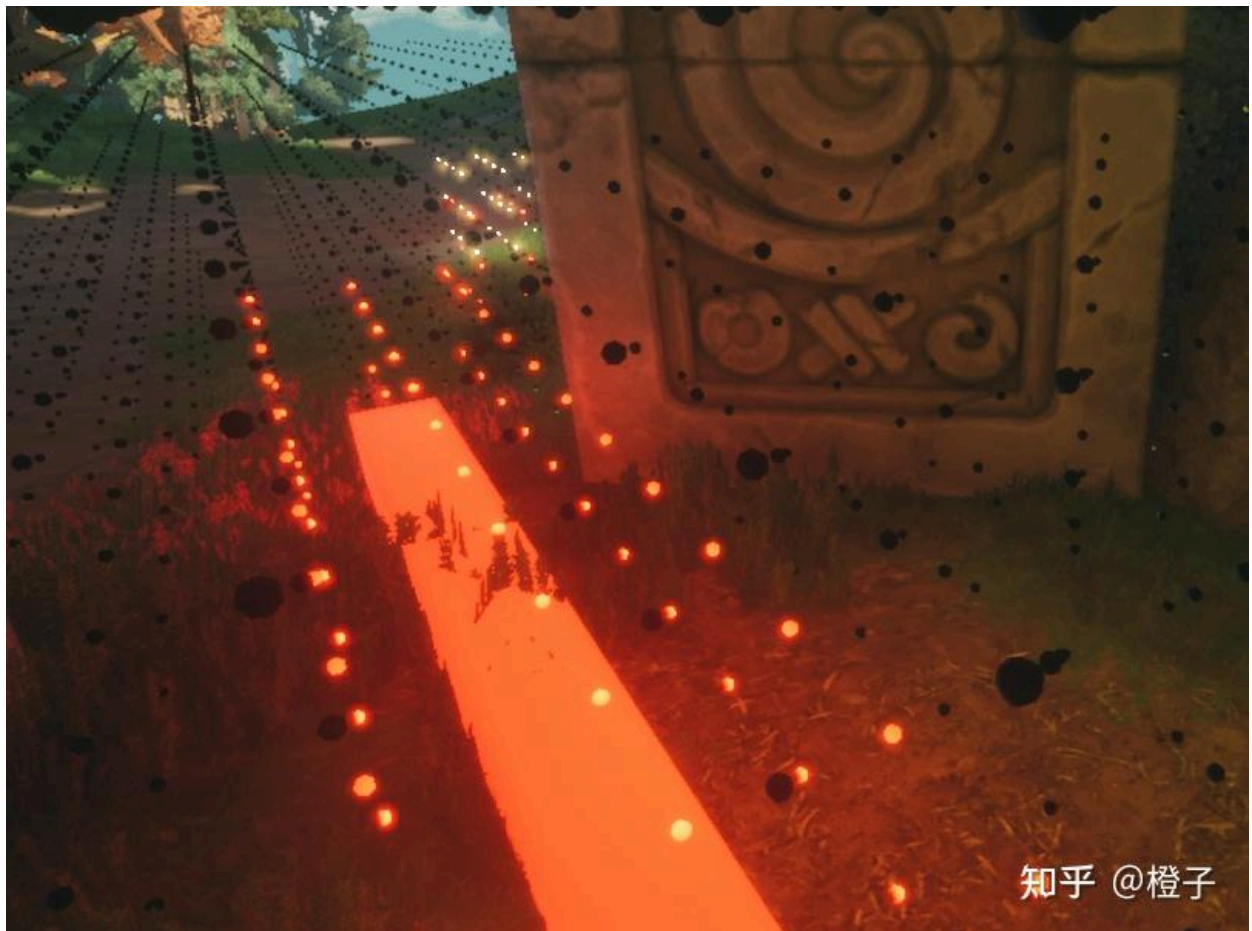


局部传输矩阵能量 >1 的情况



关闭GI的对比，我没有用点光骗人

可以看下，GI局部传输矩阵大于1时，它的Local Ref 矩阵的可视化：



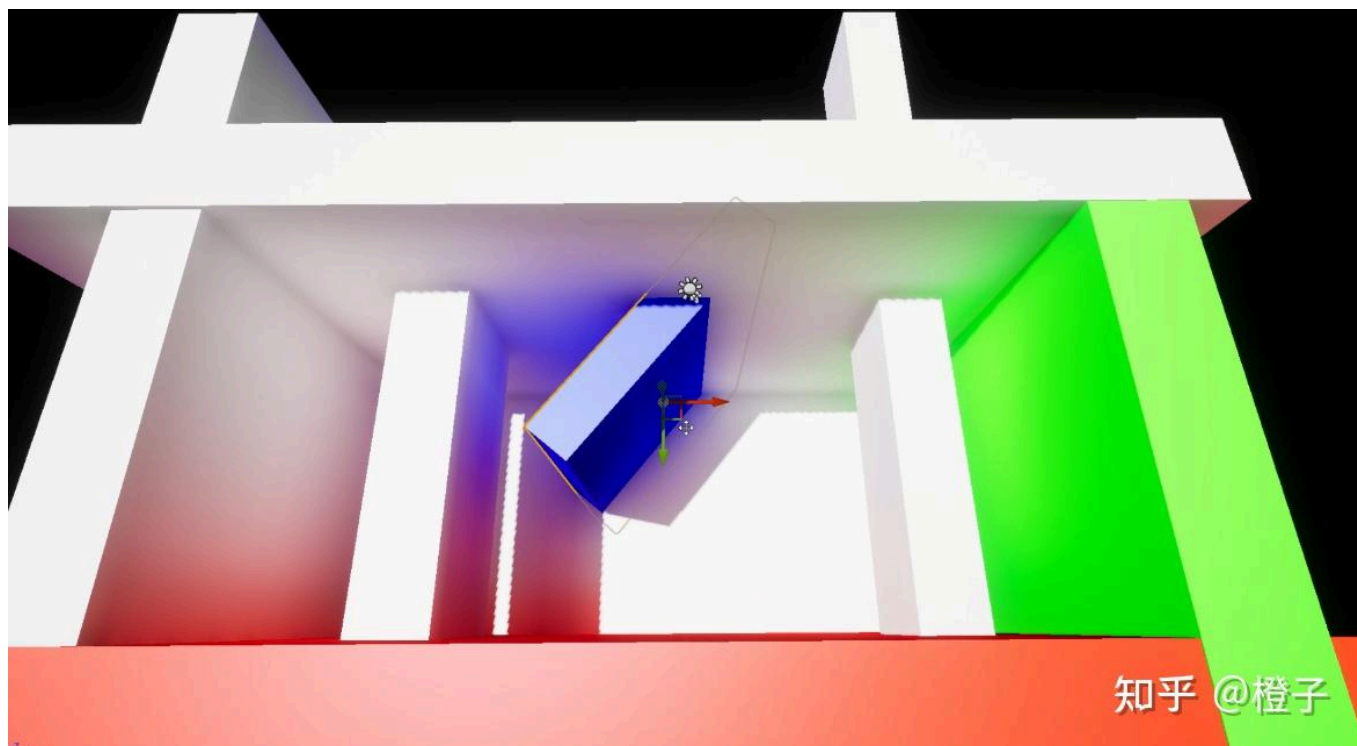
Local Ref 矩阵大于1

可以看到，它也是，发光的，表示它正在释放能量。

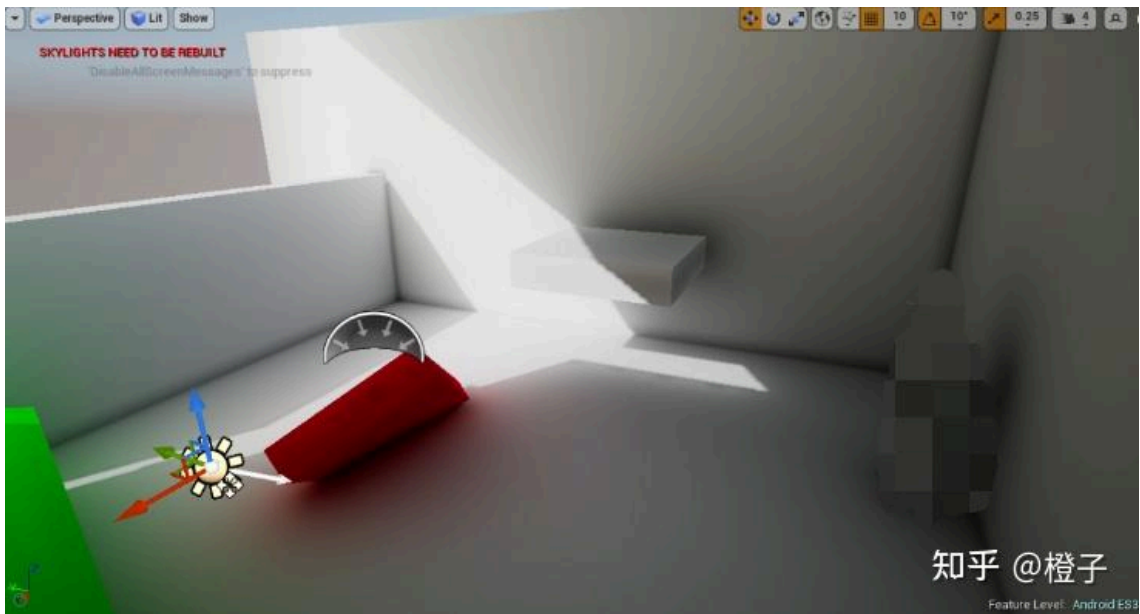
6. 尾声

至此，我们已经可以看到了整个GI算法的完整面貌了，它的核性就是快速的构造Local Transfer Matrix，并在GPU上做类Bloom的Cache友好操作做GI的传输。整个算法，从头到尾都没有Trace（Ray cast）这也是这个算法为什么这么快最核心的原因。当然由于这样，也会造成很多问题。由于这种传输算法对空间的固定划分，我们会遇到漏光这样的问题。但是本算法的漏光会略好于一些如SDF DDGI，或者其他基于Probe类的GI。因为有一份原生的Local Visibility数据，以及比较完整的Global Visibility数据。但是漏光依旧，具体的处理方法，可以依据实际情况和需求具体来选择。

其次，对于一些细腻部位的反弹效果。可以看到这个算法的表现和Bloom的密度是相关的，类似于你用多大的Bloom texture。当然Diffuse GI我们普遍认为是偏低频的数据，因此稍微糊点可能不是那么严重，但是对于一些转角处的Diffuse其实还是偏高频的。这个可以两个角度来解决，目前整个算法0.5ms，在PC 1060上，差不多是0.06ms的开销，对于Console平台，可以把密度翻倍，可以达到非常细腻的一个间接阴影和间接光照。其次可以考虑融合一下SSGI，对高频部分会很有帮助。



PC上，增加些密度，效果要好的多



PC上增加些密度也能有比较好的高频部分

最后的最后，我还是想宣传一下**胞胞机**!!!!!!!!!!!!!!!!!!!!!! 推荐大家看《A new kind of science》这本书，能给到很多启发

2023.12.21更新

GI 的实时Specular⁺ Reflection



2024.3.25 更新

今天偶然看见Youtube上一个德国人，想到了和我一样的算法。哈哈

[youtube.com/watch?...](https://www.youtube.com/watch?...)

2024.4.xx 更新

今天心情不好，更新下局部反弹，就是前面的Local Transfer Matrix 的计算伪代码。

这个是计算局部一次反弹矩阵哒！

```
//Input ColorToFill, is the voxel color, or sdf color , or one point of the local space color
float ColorToFillR
float ColorToFillG
float ColorToFillB

//SampleBasis is the direction from the center of local space to the ColorToFill
//GetSH2PIDivDFT is the function to calculate the SH basis function from a given direction. here is
the direction to the color to fill
FSHVector2& BASH = SampleBasis;
FVector4 DA = GetSH2PIDivDFT(-SampleDir);
FVector4 DABA_x = DA * BASH.V[0];
FVector4 DABA_y = DA * BASH.V[1];
FVector4 DABA_z = DA * BASH.V[2];
FVector4 DABA_w = DA * BASH.V[3];

//TransferToWriteOut is the local transfer matrix, It has three Component, RGB foreach
//in a higher level usage of TransferToWriteOut is to make it lighter.
//eg: replace the RGB of transfer matrix to a Luminance Plus Color .
//the luminance take one 4x4 matrix and the color only take 3 8bit float

TransferToWriteOut.R.Factor0 += DABA_x * ColorToFillR;
TransferToWriteOut.R.Factor1 += DABA_y * ColorToFillR;
TransferToWriteOut.R.Factor2 += DABA_z * ColorToFillR;
TransferToWriteOut.R.Factor3 += DABA_w * ColorToFillR;

TransferToWriteOut.G.Factor0 += DABA_x * ColorToFillG;
TransferToWriteOut.G.Factor1 += DABA_y * ColorToFillG;
TransferToWriteOut.G.Factor2 += DABA_z * ColorToFillG;
TransferToWriteOut.G.Factor3 += DABA_w * ColorToFillG;

TransferToWriteOut.B.Factor0 += DABA_x * ColorToFillB;
TransferToWriteOut.B.Factor1 += DABA_y * ColorToFillB;
TransferToWriteOut.B.Factor2 += DABA_z * ColorToFillB;
TransferToWriteOut.B.Factor3 += DABA_w * ColorToFillB;
```

```

//here is the division of local transfer matrix factor. weight Sum is the number of ColorToFill we take.
//if one sample, it`s one, if 26 sample, it`s 26.
//float Factor = IL_PI4 / WeightSum; // we do importance sampling here
OutProbe.LocalVisibilitySH = OutProbe.LocalVisibilitySH * Factor;

//every sample point we take into consider need to add up. and multiply the factor
// build local PRT
{
    TransferToWriteOut.R.Factor0 *= Factor;
    TransferToWriteOut.R.Factor1 *= Factor;
    TransferToWriteOut.R.Factor2 *= Factor;
    TransferToWriteOut.R.Factor3 *= Factor;
    TransferToWriteOut.G.Factor0 *= Factor;
    TransferToWriteOut.G.Factor1 *= Factor;
    TransferToWriteOut.G.Factor2 *= Factor;
    TransferToWriteOut.G.Factor3 *= Factor;
    TransferToWriteOut.B.Factor0 *= Factor;
    TransferToWriteOut.B.Factor1 *= Factor;
    TransferToWriteOut.B.Factor2 *= Factor;
    TransferToWriteOut.B.Factor3 *= Factor;
}

```

然后我们可以通过这个求得局部小空间内的无穷次反弹：

```

// The Local Reflection can be write as a inf series, while T is Local Reflection PRT Matrix
// Than :
// InfBoundT = T + TT + TTT + TTTT + TTTT ... T^n
// Follow with Neumann Law, a given A`s Neumann series :
// I + A + AA + AAA + ... + A^n = (I - A)^-1
// than we can get:
// InfBoundT = (I - T)^-1 - I
// with such matrix, we can do Inf bound of local reflection in just one matrix mul!

```

这里就不放伪代码了，本质上就是使用Neumann Law 对上述矩阵求极限。求也可以，不求也可以。直接用上面的矩阵也可以。
 之对收敛速度有微小影响。

最后，我想说说SH

如果没有实际深入使用过SH，可能很难对SH有个直观的理解。导致没法用好这个东西。

首先我们先不管那些乱起八糟的公式。

记住

1阶SH : float

2阶SH : float4

3阶SH : float9

SH就是一个球面上的函数，就像你一个2维函数⁺，3维函数一样。对于SH来说。你可以把一个分布在球面上的数据存储在SH里，然后你可以很快的用一个方向去取那个值。就像做Cubemap一样。或者，你就可以把SH当作是一个Cubemap。

SH的第0个系数，基本代表了这个SH的总能量。什么是总能量？总能量就是你球面上所有值都加起来的值。所以如何理解一个可见性SH。它的第0个系数代表了这个SH它是完全不可见，还是完全可见，还是在之间的某个状态。剩下3个分量代表了不同方向上对整个权重的分配。比如是右边更不可见，左边更可见一点还是怎样。

接下来，从做Shader的美术视角，我们就会遇到这样几个问题：

1. 怎么把Cubemap上每个点的值都加起来得到一个新的Cubemap。在SH上怎么做？
2. 怎么把Cubemap上每个点的值都乘起来得到一个新的Cubemap。在SH上怎么做？
3. 怎么算Cubemap上所有点的和。在SH上怎么做？
4. balblabl

这就像是对于SH这个空间，你有哪些操作符，这些操作符，决定了它是不是好用，能不能方便解决某个问题。

首先SH有加法，也有乘法，但是没有除法。如果喜欢离散数学，那么这个应该算是群，还是环 还是域来着？它的操作符应该构成有限单群⁺啥的？类似的意思。

SH的加法

就是直接相加。 $xyzw + xyzw = xyzw$ 。这样你就把两个cubemap加起来了。

乘法有点特殊，很多引擎和文献里都没有提到。它叫做SH Triple Product。在研究量子物理的那边，经常用。

那乘法怎么做？

```
half4 SHTripleProduct(half4 InSHA, half4 InSHB)
{
    half4 RetSH;
    RetSH.x = dot(InSHA, InSHB);
    RetSH.yzw = InSHA.xxx * InSHB.yzw + InSHB.xxx * InSHA.yzw;
```

```

return RetSH * 0.28209479f;
}

```

这是我从一个量子力学的库里面总结出来的。就一个dot 加上 两个Mad⁺

那积分(全部加起来求和)怎么做？

SH[0] * 3.5449077018110320545963349666823f; // 2 * Sqrt(PI)

这就是积分。

2024.9.1 更新

来聊一聊局部传输矩阵Local Ref Matrix 的旋转

旋转局部传输矩阵涉及应用由 Wigner D-矩阵表示的旋转算子。以下是如何直接旋转矩阵⁺ B:

直接旋转局部传输矩阵 B

概述

要在不重新构建的情况下旋转 局部传输 矩阵 B:

$$\begin{aligned}
 B' &= D^\top(R) \cdot B \cdot D(R) B' = \mathbf{D}^\top(\mathbf{R}) \cdot \mathbf{B} \cdot \mathbf{D}(\mathbf{R}) B' \\
 &= D^\top(R) \cdot B \cdot D(R)
 \end{aligned}$$

其中:

- B 是原始的 局部传输 矩阵 (4x4) 。
- B' 是旋转后的 局部传输 矩阵。
- $D(R)\mathbf{D}(\mathbf{R})D(R)$ 是对应于所需旋转 R 的 旋转算子。
- $D^\top(R)\mathbf{D}^\top(\mathbf{R})D^\top(R)$ 是 $D(R)\mathbf{D}(\mathbf{R})D(R)$ 的转置⁺矩阵，等同于 $D(R-1)\mathbf{D}(\mathbf{R}^{-1})D(R-1)$ ，因为旋转算子是正交的。

详细步骤

1. 构建 局部传输矩阵 旋转算子 $D(R)\mathbf{D}(\mathbf{R})D(R)$:

- 对于阶数 $lmax = 1l_{max} = 1lmax = 1$ 的 SH，总的 SH 系数数量为 $(lmax + 1)^2 = 4(l_{max} + 1)^2 = 4(lmax + 1)^2 = 4$ 。

- 局部传输矩阵 旋转算子 $D(R)\mathbf{D}(\mathbf{R})D(R)$ 是一个块对角矩阵

1. 计算旋转后的矩阵 $B\mathbf{B}'B'$:

- 使用公式 $B' = D^\top(R) \cdot B \cdot D(R)B' = \mathbf{D}^\top(\mathbf{R}) \cdot \mathbf{B} \cdot \mathbf{D}(\mathbf{R})B'$ 。

$$= D^\top(R) \cdot B \cdot D(R)$$
- 这实际上是通过输入和输出的 SH 系数应用旋转来旋转 $\mathbf{B}$ 描述的空间。

说人话就是，如果简单的把局部传输矩阵当作对于一个空间的描述，那么如果要旋转这个空间的出射光，需要首先对这个空间进行反向的旋转，这样我们可以从正确的位置接受入射光，其次我们需要把出射的光再正着旋转出去。因为这里用到的旋转矩阵就是普通的旋转矩阵，因此它的转置和它的逆等价。所以先乘旋转矩阵的转置，再乘局部传输矩阵，最后乘旋转矩阵。

当然，能这样算也是因为目前是SH2，恰巧有这种性质。如果是SH3的局部传输矩阵，旋转会异常复杂。

编辑于 2024-12-30 10:16 · 广东

手游社区

计算机图形学

移动端开发

来源：知乎 | 悠趣谷·零成本 | 我要插件